

AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Topic 4 – Async Research Assistant

Research Assistant Team

Ruslan Sadigov • Nihat Ismayilzade • Samur Eyyubov • Polad Ibrahimli

Final Project Defense

`github.com/ruslan-sadigov/research-assistant-SWE-Final-Project`

The problem in one sentence

What the system does

A user asks a research question; the tool collects evidence from Wikipedia, arXiv and web search at the same time and prints an LLM answer whose numbered citations point to that evidence. A failing source becomes a warning, not a crash.

Inputs

- ▶ Question text, up to 2000 characters
- ▶ Sources: any of wiki, arxiv, web
- ▶ Optional `--no-cache`; keys and limits from `.env`

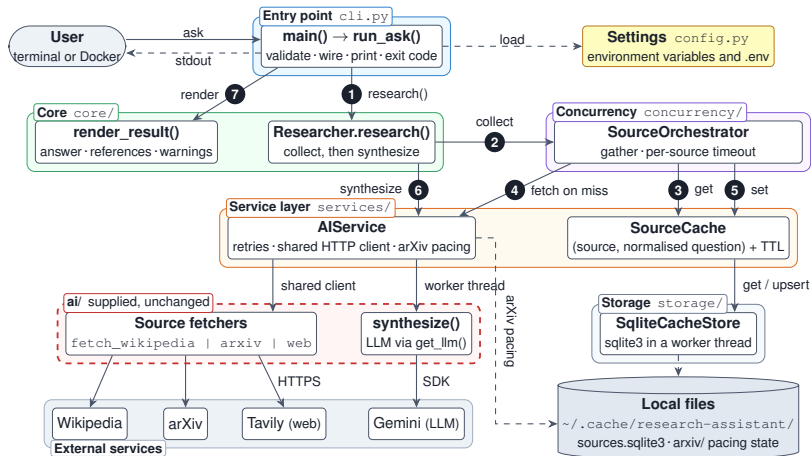
Outputs

- ▶ Answer, references, per-source warnings
- ▶ Exit code 0 (answer), 1 (none), 2 (invalid input)
- ▶ Source evidence cached in SQLite for 24 h

Entry point: CLI only, no HTTP server: `python -m researcher ask "..."`
`--sources wiki,arxiv,web`

Architecture

Module map. The boundary between the *provided* ai/ package (red, dashed) and *our* SE layer is shown explicitly.



Solid arrows: calls, numbered in request order. Dashed: configuration, local files, output.

Technology stack

Layer	Technology	Why this one
Language, runtime	Python 3.12, asyncio	<code>gather</code> , <code>timeout</code> and <code>to_thread</code> are in the standard library
Environment	uv with a locked <code>uv.lock</code>	Same versions locally, in CI and in Docker
Config, models	<code>pydantic-settings</code> , <code>python-dotenv</code> , <code>Pydantic v2</code>	Typed settings and self-validating boundary types
HTTP	<code>httpx</code> async client + our <code>RetryingTransport</code>	One shared client; retries and pacing in one place
Storage	SQLite (standard <code>sqlite3</code>)	No server to run; the file survives restarts in a volume
Providers (in ai/)	Wikipedia, arXiv, Tavily, Gemini	Supplied package; chosen by environment variables
Tests	<code>pytest</code> , <code>pytest-asyncio</code> , <code>pytest-cov</code> , <code>MockTransport</code>	Async tests, offline, with coverage
Quality gates	Ruff, Black, isort, mypy	One style and typed signatures across four authors
Delivery	GitHub Actions, multi-stage Docker	Lint, types, tests and an image on every push

Everything except the `ai/` package and its providers is our own code and configuration.

Three design decisions we can defend

1. **asyncio over threads, one deadline per source**

All work is waiting on at most three sources, so one event loop is enough.

Rejected: threads, and TaskGroup, which cancels the other sources when one fails.

2. **Protocols and constructor injection around the unchanged ai/**

Tests swap in fakes without patching. Trade-off accepted: one wiring function (`run_ask`) must build everything in the right order.

3. **SQLite cache of source evidence, not of answers**

Every run still synthesizes; repeated questions skip the network. We would change it for many parallel users (single-file writer) or if answers became expensive.

Expect to be asked

“Why did you not use threads / a semaphore / Redis / an answer cache?” Have a 15-second answer ready for each of the three.

Concurrency: the benchmark

Workload: the 5 supplied questions $\times$ all 3 sources, cache off. Offline: simulated delays 0.1 / 0.2 / 0.3 s, median of 5 runs. Live: Wikipedia, arXiv, Tavily, one pair per question.

	Sequential	Concurrent (gather, ≤ 3 tasks)
Offline wall-clock (s)	0.623–0.628	0.312–0.314
Offline speedup	1.0 $\times$	1.99–2.00 $\times$
Live wall-clock (s)	4.41–7.04	3.62–5.60
Live speedup (q1–q5)	1.0 $\times$	0.79–1.82 $\times$
Failed or timed-out sources	0	0

Bottleneck named: the slowest source sets the concurrent time (0.3 s offline); live, network latency and arXiv's 3 s request spacing.

What didn't reach $N\times$: with unequal delays the ideal is 2 $\times$, not 3 $\times$; live, q2 was slower concurrently and Wikipedia found nothing for full questions.

Reproduce (README): `uv run python benchmarks/benchmark_sources.py --repeats 5`

Robustness: one concrete failure mode

Scenario: in a live run with all three sources, arXiv timed out.

How we surfaced it: manual live CLI run; reproduced offline in `test_deadline_cancels_slow_fetch_and_preserves_success`.

What the system does:

- ▶ **Retry:** up to 3 attempts, 0.5 s growing to 4 s, only for timeouts, 408, 429 and 5xx; Retry-After honoured; one 10 s deadline per source.
- ▶ **Validate:** input and settings checked before any call (exit 2); models reject unknown fields; cached rows re-validated on read.
- ▶ **Log:** source timeout: `source=arxiv stage=... deadline_expired=...`; failures log the error type only, no keys or messages.
- ▶ **Degrade:** Wikipedia's evidence was kept and the answer was printed with an arXiv warning; exit code 0.

Also seen live: intermittent Gemini 503 (high demand); later calls succeeded.

Caching: measured, not assumed

Schema and policy

- ▶ Table `cache_entries`: key (source, query_key), its automatic unique index, JSON payload; no foreign keys
- ▶ Key = source + normalised question; TTL 24 h
- ▶ All rows derived; the external sources stay authoritative
- ▶ `--no-cache` never opens the SQLite file

Staleness: empty results are also kept for 24 h, and the key ignores provider settings.

Reproduce: `uv run python tests/cache_verification.py` (*docs/cache-verification.md*)

Hit rate = hits / lookups

First run	0 / 15
Second run	15 / 15
Combined	15 / 30 = 50%

5 questions × 3 sources through the real CLI; providers simulated; an answer synthesized on every run.

Live demo (or scripted demo)

What we will run

One question against all three sources, twice, with a cache volume: the second run is served from SQLite and still produces an answer.

Demo command (works inside Docker):

```
docker run --rm --env-file .env --env DATABASE_URL= `
-v research-assistant-cache:/home/appuser/.cache/research-assistant `
research-assistant:runtime ask "Photovoltaic effect" --sources wiki,arxiv,web
```

- ▶ On screen: numbered references from all three sources; on the second run, `cache_hit=True` in the log.
- ▶ Timing: in a live container run the sources took 2.39, 3.53 and 0.53 s, yet collection finished in 3.64 s instead of 6.45 s.

Backup: pre-recorded screencast, then the offline run `uv run python tests/cache_verification.py`.

Testing

Coverage & shape

- ▶ Total coverage: 98.44 %
- ▶ Provided ai/ smoke tests: **passing**
- ▶ Test count: 267 own + 16 smoke
- ▶ All tests offline; AI module fully mocked

Three tests worth calling out

- ▶ **Happy path:** the real CLI answers each supplied question, reuses SQLite, then honours `--no-cache`.
- ▶ **Concurrency:** arXiv raises, gather degrades; Wikipedia and web evidence kept.
- ▶ **Error path:** HTTP 408/429/500/503 is retried once, then succeeds; every response is closed.

Tooling

`pytest + pytest-asyncio + pytest-cov + httpx.MockTransport`. mypy: no issues in 15 source files; Ruff, Black and isort clean in CI.

Limitations & what we'd do next

Where the system is brittle today

- ▶ Full questions often find no Wikipedia match; there is no query rewriting.
- ▶ No fallback LLM: if Gemini is down, we show evidence but no answer.
- ▶ The cache keeps empty results for 24 h, ignores provider settings, and never evicts rows.

Given another week we would

1. Turn the question into a short topic query before searching Wikipedia.
2. Add a settings fingerprint to the cache key, a shorter TTL for empty results, and a purge command.
3. Repeat the live benchmark at least three times and compare distributions.

Team contributions

Member	Primary ownership	Commits
Ruslan Sadigov	Settings, models, interfaces, orchestrator, retry transport, arXiv pacing; merges and release	53 (65.4%)
Nihat Ismayilzade	AIService around ai/; engineering audit, tests, Docker and provider docs	6 (7.4%)
Samur Eyyubov	SourceCache and SQLite store; architecture diagram, cache veri- fication, slides, demo	12 (14.8%)
Polad Ibrahimli	CLI and Researcher; report, README, contribution statement	10 (12.3%)

Commits on main, 18 Sep 2026, 81 in total, author aliases merged. Final shares: signed contribution statement.

We can all answer:

Any of us can walk through any module on demand. Pick a file — we'll defend it.

AI tool use: disclosed per member in the contribution statement; e.g. Claude drafted Samur's architecture figure, cache tests and this deck, which Samur reviewed and ran.

Questions?

Research Assistant Team

`github.com/ruslan-sadigov/research-assistant-SWE-Final-Project`

“We can walk through any file in the repo.”